Universität Siegen

Naturwissenschaftlich-Technische Fakultät

Department Elektrotechnik und Informatik

Lehrstuhl für Regelungs- und Steuerungstechnik

Studienarbeit

<<UPGRADING AND RECOMMISSIONING OF THE TRACKED ROBOT>>

von / by

<<Monish Senthil Kumar>>

Matrikelnummer / Matriculation number

<<1700943>>

Prüfer / Examiner

Prof. Dr.-Ing. habil. Michael Gerke

2. Prüfer / Examiner

M.Sc. <<xx>>

Tag der Abgabe / Day of delivery

<<Date>>

Chapter 1: Introduction

- 1.1 Project Background
- 1.2 Objectives of the Project
- 1.3 Scope of Work
  - 1.3.1 Mechanical and electrical inspection
  - 1.3.2 Control system development
  - 1.3.3 Integration of sensor systems
  - 1.3.4 Software execution
  - 1.3.5 System check and verification

Chapter 2: Hardware System Architecture

- 2.1 Robot Recommissioning Overview
- 2.2 Hardware Components
  - 2.2.1 Arduino Mega
  - 2.2.2 Hall Sensor
  - 2.2.3 Motor Drivers
  - 2.2.4 Encoders
  - 2.2.5 Motors
  - 2.2.6 Brakes
  - 2.2.7 Mosfet
  - 2.2.8 Ultrasonic Sensor
  - 2.2.9 Terminal Blocks
  - 2.2.10 DC-DC Converter
- 2.3 Wiring and Connections
- 2.4 Hall Sensor mount design
- 2.5 Communication Interfaces

Chapter 3: Control System

- 3.1 PID Control for Speed Regulation
- 3.2 Encoder Integration for Angle Calculation
- 3.3 Frequency of an Encoder Signal
- 3.4 Motor Movement Logic

Chapter 4: Implementation and Setup

- 4.1 System Setup and Configuration
- 4.2 Arduino setup () Function
- 4.3 Arduino Loop () Function

Chapter 5: Conclusion and Future Work

- 5.1 Summary of Work Done
- 5.2 Recommendations for Improvements
- 5.3 Future Enhancements

ABSTRACT

This project work focused on the enhancement and recommissioning of an already existing tracked robot platform to increase performance, reliability, and reproducibility for research and practical applications. The aim of the project was to enhance the performance of both the hardware and software subsystems to stabilize and enhance the usability. In particular, the initial effort to improve the hardware subsystem included new motor drivers, ultrasonic sensors, Hall Effect sensors, and Encoders that would provide better resolution for speed regulation and perception of the environment.

The upgraded software was high level control features that incorporated encoder based PID regulation, home functions with Hall sensors, and I²C communication to allow the robot with an Arduino Mega 2560 to perform actions on peripherals. Improved functionality also required the design, build, and upgrade of the flipper arm mechanism, specifically improved position control and braking mechanisms, to allow operability against uneven terrain.

The recommissioning of the robot involved taking it into an operational state and then upgrading and incorporating the upgrades so the robot was prepared for future upgrades, especially with respect to autonomous navigation and the integration of higher order control methods. The evidence from the work was that serial upgrades to existing legacy robotic platforms, could reduce maintenance and improve operational usability while providing an opportunity for future adaptable research and work.

1. INTRODUCTION

1.1 PROJECT BACKGROUND

In recent years, mobile robotics has seen a massive improvement in the fields such as the defence sector, emergency rescue operations, agricultural tasks, and in the automation industries. Compared to all the robotic platforms, the tracked robots give better control while operating in uneven terrains because these tracked robots have more ground contact and more stability. Since robot technology is developing day by day, the older tracked robots are also being updated in terms of control systems, energy efficiency, and their performance.

My project focuses on upgrading and recommissioning an existing tracked robot, and the task is to improve the control systems and make them simpler regarding wiring and connections. The tracked robot has flipper arms in the front on both sides that are used for overcoming obstacles, and it has two continuous tracks for forward and backward movements.

This upgrade aims to develop and integrate a performance-related control system that is more capable of delivering a speed control regulation for the robot's movements. Also, it handles positioning to find the home spot for the flipper arms. By implementing these control systems on the robot, the tracked robots should be capable of performing smooth movements in different conditions.

To technically improve and be cost-effective, the existing hardware, actuators, sensors, and integration of new control systems will upgrade the tracked robot. This project allows for applying theoretical knowledge by merging the concepts of robotics, programming, control systems, and real-world validation, making this project suitable for both academic and practical experience.

The main goal is to completely transform the robot into possibly a highly intelligent and reliable mobile robot that can execute the exact position control and speed control for the movements, including the flipper arms, while maintaining consistent performance across various working conditions.

By implementing all the upgrades to the tracked robots for both software and hardware, the tracked robot will be capable of future upgrades like obstacle detection, step climbing, wireless communication, and autonomous navigation. These kinds of developments in this robot pave the way for the capability of the tracked robot to handle real-time tasks as well. These recommissioned robots will be the foundation for the advanced robotic systems in the automation industries.

1.2 OBJECTIVES OF THE PROJECT

The fundamental objective of the project is to convert a tracked robot that is non-functional or semi-functional into a fully functioning and upgraded platform capable of completing movements autonomously, as well as controlled mobility. The objectives below outline the goals of the project, and what we hope to achieve.

- **To implement and design a reliable speed control system** for all the tracked robot movements, including forward, backward, left turn, and right turn.
- **Integrate and develop the position control mechanisms** for the flipper arms, which allow them to reach the desired angular position for support**.**
- **To achieve smooth angle control** for the flipper arms, which suits the corresponding work environment.
- **To test and integrate the required sensors and the feedback systems,** such as optical encoders and Hall sensors, to enable control and state monitoring.
- **To recommission and upgrade the complete system** by validating its movements, tuning the control parameters, and checking its stable performance under real-world conditions.
- **To document the complete upgraded process,** including designs, hardware, software implementations, and results.

1.3 SCOPE OF WORK

The primary objective of this project is to improve hardware and software, conduct the systems assessment, and document. The systems assessment expected to take place is outlined below:

1.3.1 MECHANICAL AND ELECTRICAL INSPECTION

- Check the condition of all the mechanical parts, belts, and the flipper arms.
- To conduct a diagnostic analysis of the motor actuation, sensor input, and driver response to check the system's robustness and control systems alignment.
- Inspect and replace wiring, connectors, and terminals for a safe and adequate power supply and signal flow.

1.3.2 CONTROL SYSTEM DEVELOPMENT

- Developing a **speed control** for the tracked movements from the feedback that we get from the Encoder.
- Designing and coding the **control algorithms** to obtain precise flipper arm movement, speed, and position.
- A suitable **communication protocol**, like the I2C, is used between the microcontroller (like Arduino Mega) and peripheral components, such as motor drivers, sensors, and brakes.

1.3.3 INTEGRATION OF SENSOR SYSTEMS

- Installed and wired the wheel encoders for the robot's velocity calculation and feedback control.
- Installed the Hall Effect sensors to identify the homing positions for the flipper arms.
- Therefore, ingesting the ultrasonic sensors' inputs to interpret real-world behaviours.

1.3.4 SOFTWARE EXECUTION

- The embedded software code was executed to control all tasks in real-time using Arduino IDE.
- The control systems were analysed and validated using the serial monitor/plotter.

1.3.5 SYSTEM CHECK AND VERIFICATION

- Checked individual system validation in a variety of trials, which included:
    - Moving the flipper arm to a variety of angles.
    - Ensure the speed performance of the robot stands against variable load performance.
- Validation against a complete performance of the system in ground testing.

2. HARDWARE SYSTEM ARCHITECTURE

2.1 ROBOT RECOMMISSIONING OVERVIEW

Recommissioning the tracked robot will entail restoring functionality through hardware restorations and upgrades to the control algorithm.

The tracked robot is equipped with continuous tracks for movement on each side and flipper arms for obstacle avoidance. Many of the upgrades will consist of the I2C communication, installation of the hall sensors for homing, replacement of the motor drivers, and replacement of the ultrasonic sensors.

The existing structure of the tracked robot will be reconditioned, but the only things being changed are the wiring, the microcontroller, and a couple of the sensors. These changes will help make the tracked robot a fully functional robot for everyday tasks in the real world and ensure stability for smooth movements.

2.2 HARDWARE COMPONENTS

The hardware system has several components that are combined to work together to complete their tasks. In this project, the hardware components that played a significant role are the microcontrollers, like Arduino Mega 2560, Sensors, like TLE4905L hall sensor, SRF02 ultrasonic sensor, MD03, MD04 motor drivers, ME22 encoders, electrical brakes, DC to DC converters, and terminal blocks with fuses for a smooth power supply.

2.2.1 ARDUINO MEGA

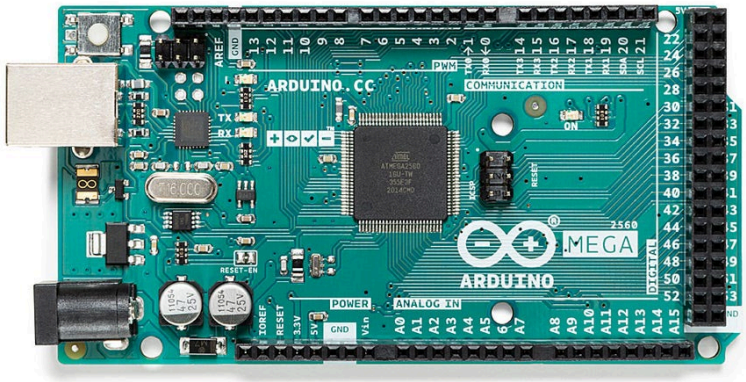


Figure 2: Arduino Mega 2560 [1]

Arduino Mega 2560 is an **open-source microcontroller** development board based on the ATmega2560 microcontroller. It is commonly used in embedded systems, robotics, and automation projects as it has reasonably good input/output (I/O) pins, memory, and communication interfaces. For this tracked robot project, the Arduino Mega 2560 will be the **central control unit** managing to interface between all hardware components, process data from sensors, and execute control algorithms to govern the behaviour of motors and actuators.

Arduino Mega 2560 has a specific capability: its input/output processing power. Arduino Mega 2560 provides **54 digital input/output pins**. Of these 54 pins, 15 are PWM capable, 16 are analog input pins, and 4 are UART pins. The number of I/O pins is perfectly suitable for this project, where all of the above-mentioned hardware can be driven and processed simultaneously. This will allow us to omit the expanders and simplify the wiring.

The board is clocked at 16 MHz, sufficient for handling real-time control operations, especially when optimized code and efficient interrupt handling are used. In this upgraded tracked robot, the Arduino Mega receives inputs from rotary encoders attached to the drive motors to calculate the actual movement speed. It also reads Hall Effect sensors placed on the flipper arms to detect home positions and ensure accurate positioning. Arduino Mega can also implement the PID control algorithms to control the motor speeds of the tracked robot movements in real time, ensuring smoother operations.

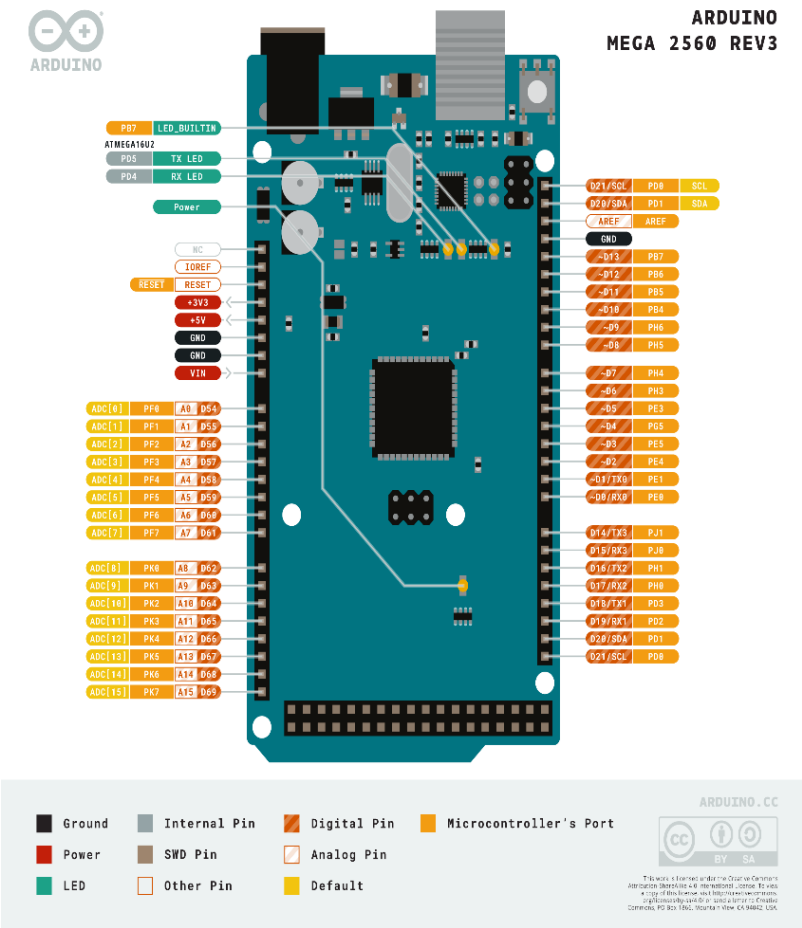


Figure 2: Arduino Mega 2560 pinout [1]

The board communicates with the interface devices using I2C, SPI protocols, and standard digital/analog lines. For example, the ultrasonic sensors connected via I2C allow the robot to detect obstacles and make a flipper decision. The motor drivers are also connected via I2C communication, which enables faster response for precise speed control of each motor attached to the tracked robot. A serial communication connects to an external human-machine interface or a computer.

The other reason we selected the Arduino Mega 2560 was its ease of programming. It has Arduino IDE support for programming using C/C++ syntax and many libraries available to integrate sensors, perform PID control programming, and serial communications. The Arduino Mega has a flash card capacity of 256KB, an SRAM memory of 8KB, and an EERPROM memory of 4KB, which allows the Arduino Mega to manage complex programs, multitask, and control motors.

Powering the Arduino Mega 2560 is flexible. It can be powered via USB when developed or using an external voltage supply or wall charger (7 -12V) during deployment with its on board voltage regulator. This robot was powered using the DC-DC converter in the robot's power architecture. This feature is used to avoid unnecessary voltage spikes introduced to the system.

The Arduino Mega is the brain of the control system microcontroller of the tracked robot. It ensures communication between the tracked robot's electrical and mechanical subsystems. Its open-source nature, extensive developer community, and proven reliability make the Arduino Mega a viable option to refit Robotics platforms such as this tracked robot.

2.2.2 HALL SENSOR

The TLE4905L is one of the magnetic field switch that is designed especially for industrial applications. Reverse polarity protection is also included on-chip as is its output protection against negative voltage transients. Based on the Hall Effect principle, this sensor detects changes in magnetic fields and converts them into digital signals. In this project, the TLE4905L is more important because it identifies the homing position of the flipper arms.

Figure 2: Hall Sensor – TLE4905L [2]

The TLE4905L is a unipolar Hall switch, meaning it only reacts to one polarity of a magnetic field, typically the magnet's South Pole [2]. When the sensor gets a magnetic field within its range of changeover, it changes the output from high to low. When the magnetic induction drops below a threshold value (due to movement of the magnet away from the sensor), the output returns to a high. This hysteresis performance gives excellent noise immunity and will help prevent false switching due to small field fluctuations, electrical noise, etc.

The main advantage of the TLE4905L is its design and its functionality. This type of Hall Sensor operates on a voltage range from 4.5V to 24V and has an open-drain output. If needed, the sensor can be pulled up with an external resistor and can be directly read via the digital pin in Arduino Mega 2560.

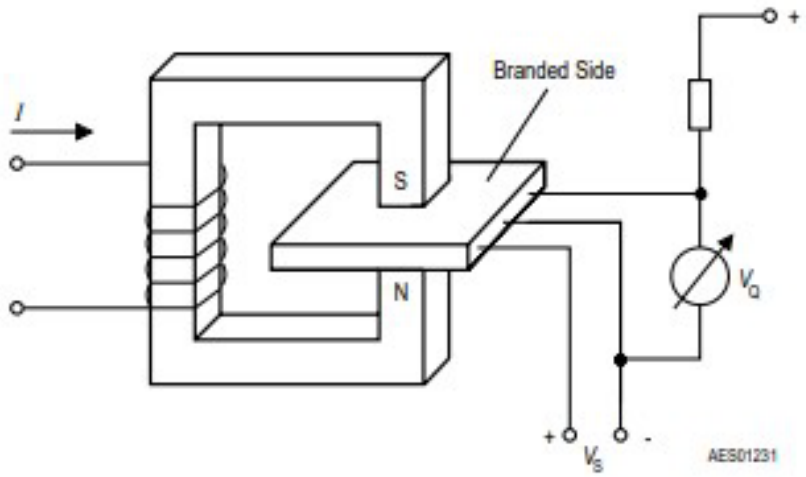


Figure 2: Hall Sensor – TLE4905L Magnetic Field Configuration

When the positive magnetic field is applied in the direction shown in the figure and the turn-on magnetic induction is exceeded, the Hall Effect output will be conducted. When the magnetic field is decreased, it will turn off. [2]

In this robotic application, a small permanent magnet is attached to the rotating shaft or moving part of the flipper arm, so that as the arm returns to the start 'home' position, the magnet would line up with the Hall sensor and output the digital signal. The low power consumption and durability of the TLE4905L also give a longer life span to the system and better use of battery resources, which is important in mobile robots since power conservation is always a concern.

2.2.3 MOTOR DRIVERS

Motor drivers are critically important intermediaries when we consider taking signals from microcontrollers to the DC motors. They enable control of the speed of the actual DC motors and facilitate the connection between greater power (high current) for the DC motors and lower power (low current) for the control signals. The DC motor drivers receive signals using an I2C connection so that the motors will operate smoothly.

In this project, we are using two kinds of motor drivers. Devantech offers the MD03 and MD04 motor drivers. The MD03 and MD04 motor drivers have been reliable with integration into Arduino systems, support high current usage, support modes of analog and digital usage, so they are very flexible.

2.2.3.1 MD03 MOTOR DRIVER

The MD03 is a high-performance motor driver and can control brushed DC motors with maximum current ratings of up to 20 Amps and maximum voltage ratings of 24V. The MD03 is explicitly designed for use with larger robotic platforms and implements either speed and direction control using analog input or digital control (via I2C). The MD03 is used in this robot system to control the motors for the flipper arms, which require tight angular positioning, and motor torque for climbing and stabilizing the structure as it moves over uneven terrain. [3]

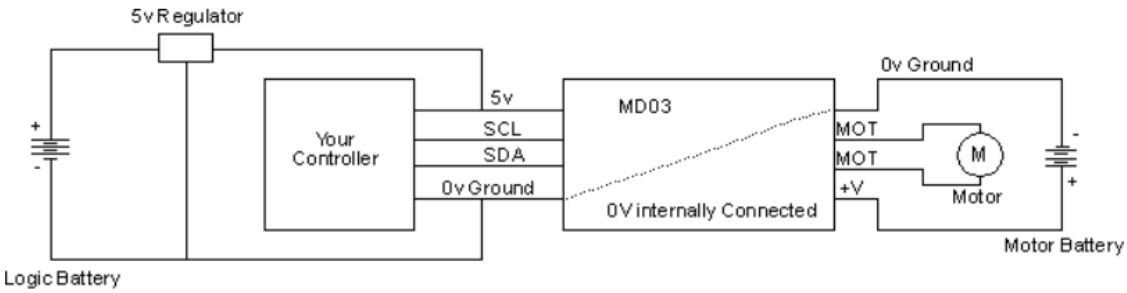


Figure 2: Motor Driver- MD03 connection [3]

The MD03 supports closed-loop speed control, which is ideal for integration with encoders. In the robot, this ensures the flipper arms reach their target positions with accuracy, even under external disturbances or mechanical resistance.

Register Address	Name	Read/Write	Description
0	Command	R/W	Write 01 Forwards - 02 Reverse (00 for instant stop - Rev9 firmware only)
1	Status	Read only	Acceleration, Temperature and Current Status
2	Speed	R/W	Motor Speed 0-255 (0x00 - 0xFF)
3	Acceleration	R/W	Motor Acceleration 0-255 (0x00 - 0xFF)
4	Temperature	Read only	Module temperature
5	Motor Current	Read only	Motor Current
6	Unused	Read only	Read as zero
7	Software Revision	Read only	Software Revision Number- Currently 9

Figure 2: Motor Driver- MD03 registers [3]

The motor driver MD03 has special registers to communicate with them via the I2C communication protocol. To read the MD03 registers, first we have to send a start bit, then the register number, where MD03 has 8 registers, as shown in the tabular column. Each address has a specific register like speed, acceleration, command, and status registers, and so on. In this project, the **MD03 motor drivers** are used to control DC motors that **operate the flipper arms, which are attached to the electrical brakes**.

2.2.3.2 MD04 MOTOR DRIVER

The MD04 motor driver is reliable in this project; the **main task** of this motor driver is to manage the **operations of the tracked robot's movements**. It delivers up to 5 Amps and can withstand up to 24 V, which makes it suitable for high-load applications. Like the MD03, the MD04 also supports multiple control methods, including PWM and I2C, and provides direction control pins for easy interface with Arduino platforms.

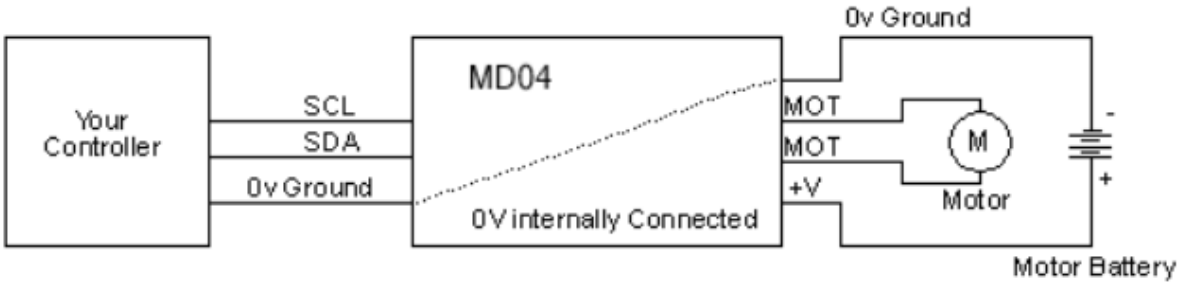


Figure 2: Motor Driver- MD04 connection [4]

The MD04 motor driver looks different from the MD03 motor drivers, where MD04 has an inbuilt power supply for the control signals. The MD04 motor driver is also a medium-power motor driver designed to supply power to lower-power H-bridges.

Register	Name	Read/write	Description
0	Command	R/W	Write 1 for forwards, 2 for reverse, 0 for instant stop
1	Status	R	Acceleration, temperature and current status
2	Speed	R/W	Motor speed 0-255 (0x00 - 0xFF)
3	Acceleration	R/W	Motor acceleration 0-255 (0x00 - 0xFF)
4	Temperature	R	Module temperature in degrees centigrade
5	Motor current	R	Motor current
6	Unused	R	Read as zero
7	Version	R	Software revision

Figure 2: Motor Driver- MD04 register [4]

The motor driver MD04 has special registers to communicate with them via the I2C communication protocol. To read the MD04 registers, first we have to send a start bit, then the register number, where MD04 has eight registers, as shown in the tabular column. Each address has a specific register, like speed, acceleration, command, status registers, etc. For forward movement, the user has to write it as 1, 2 for reverse, and 0 to stop instantly.

2.2.4 ENCODERS

From the perspective of the robotic systems, the Encoder is the most important hardware for determining the angular position, speed of the motors, and their directions. In this project, we have used two types of incremental encoders for the tracked robot, HEDS-5540-C11 and ME22-150-4.00-2LS, and they both have critical feedback functions with their corresponding systems. HEDS-5540-C11 is used to control the flipper arms' movements, and ME22-150-4.00-2LS is used to control the movement of the tracked robot.'

2.2.4.1 HEDS-5540-C11 ENCODER

The HEDS-5540-C11 is an incremental optical encoder manufactured by Avago Technologies (formerly a division of Hewlett-Packard). Within the family of HEDS-5500/5540 and HEDM-5500/5540, it is recognized for its reliability and ease of use. The HEDS-5540-C11 features a three-channel output: two quadrature signals (Channels A and B), and a third channel called the Index channel (Channel I), which can be used to set a reference or "home" position every time the Encoder makes a complete rotation.

Figure 2: Encoder: HEDS-5540-C11 [5]

The Encoder has a patterned disk that turns between an LED and a photo sensor [5]. The disk has alternating opaque and clear sections to allow or block light from the LED. As the disk turns, the sensor detects changes in blocked light to sensed light, producing digital signals based on detected movement of the position of the shaft.

The digital signals are phase-shifted 90° (quadrature), allowing both direction and speed detection of the shaft or rotational device being detected. The third output channel is the Index channel, which sends a short high signal every time the incremental output makes a complete rotation.

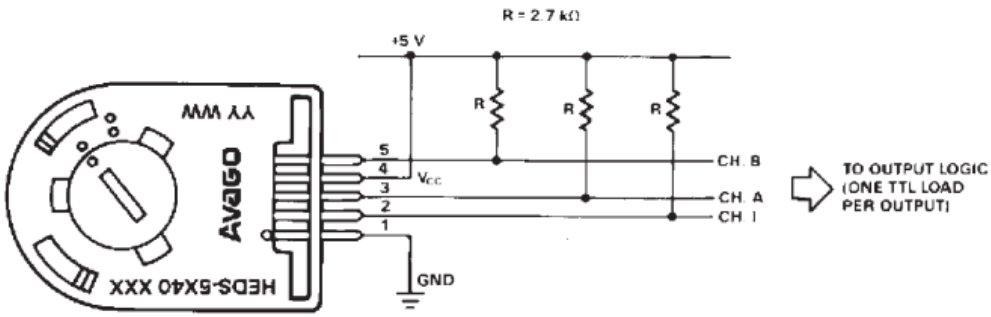


Figure 2: Encoder: HEDS-5540-C11 Pinout [5]

The **Encoder is mounted** directly onto the **flipper arm actuator's shaft** in this project. The continuous encoder signals from Channel A feed into an interrupt-driven counter that tracks angular displacement in real time. The interrupt pins are **18** and **19** in the Arduino Mega for the both encoders Channel A. This model has 100 counts per revolution (CPR).

		HEDS 5500	HEDS 5540	HEDM 5500	
Pulses per revolution	N	96 - 512	100 - 512	1,000 -1,024	
Signal output (quadrature)		2	2+1 index	2	channels
Supply voltage	V_{cc}	4.5 to 5.5			V DC
Current consumption, typical (V_{cc} = 5 V DC)	I_{cc}	17	57	57	mA
Pulse width	P	180 ± 45	180 ± 35	180 ± 45	°e
Phase shift, channel A to B	Φ	90 ± 20	90 ± 15	90 ± 15	°e
Logic state width	S	90 ± 45	90 ± 35	90 ± 45	°e
Cycle	C	360 ± 5.5	360 ± 5.5	360 ± 7.5	°e
Signal rise/fall time, typical	tr/tf	0.25 / 0.25			µs
Frequency range [1]	f	up to 100	up to 100 [2]	up to 100	kHz
Inertia of code disc	J	$8.497 \cdot 10^{-6}$			oz-in-sec²
Operating temperature range		– 40 to +100 (– 40 to +212)		– 40 to +70 (– 40 to +158)	°C (°F)

Figure 2: Electrical data of HEDS-5540-C11 [5]

2.2.4.2 ME22-150-4.000-2LS ENCODER

The movement of the tracked robot is monitored using the ME22-150-4.000-2LS encoder, developed by PWB Encoders GmbH. This model belongs to the ME22 series, compact, low-cost hollow shaft encoders designed for easy integration into compact robotic systems.

The ME22-150-4.000-2LS delivers two-channel quadrature output using TTL-compatible logic levels, and is powered by a 5V supply. It has internal pull-up circuits and a 5-pin connector for the signal output. Here, ME22-150 represents the **150 counts per revolution (CPR).**

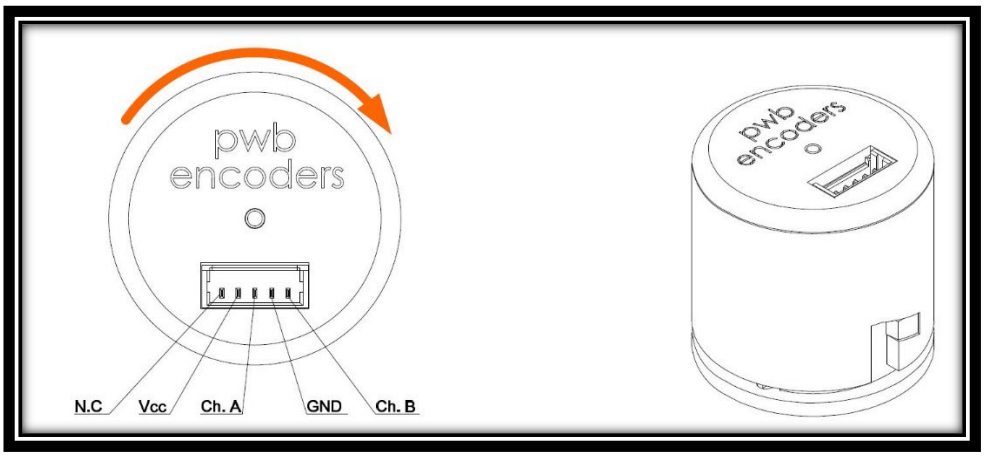


Figure 2: Encoder: ME22-150-4.000-2LS Pinout [6]

One of the significant strengths of the ME22 encoder is its compact form factor (22.0 mm diameter × 21.9 mm length) and its support for shaft diameters up to 9.525 mm, which makes it ideal for integration into the confined space of a tracked robot chassis. The Encoder should not be reassembled once it is installed. [6]

Parameter	Symbol	Min.	Typ.	Max.	Unit	Notes
Operating temperature	T_A	-20	25	85	°C	
Supply voltage	V_{cc}	4.5	5.0	5.5	V_{DC}	
Supply current (two channels)	I_{cc}	13	15	18	mA	
Supply current (three channels)	I_{cc}	33	35	38	mA	
Load capacitance	C_L			100	pF	internal pull-up 2.7 kΩ
Count frequency	f			60	kHz	rpm x N / 60 x 10^{-3}
A & B Channel						
High level output voltage	V_{oH}	2.4		Vcc	V_{DC}	I_{oH} = -0.2 mA
Low level output voltage	V_{oL}			0.4	V_{DC}	I_{oL} = 8.0 mA
Rise time	t_r		500		ns	C_L = 25 pF;
Fall time	t_f		100		ns	R_L = 2.7 kΩ

Figure 2: Electrical data of ME22-150-4.000-2LS [6]

In this robot, the ME22-150-4.000-2LS is integrated with the main drive motors that propel the left and right tracks. The robots' on board microcontroller can calculate both speed and travelled distance by reading the Encoder signals in real-time through external interrupts or timers. Both encoders, HEDS-5540-C11 for the flipper arms and ME22-150-4.000-2LS for the tracked base, play distinct yet complementary roles in the robot's motion control system. The ME22 encoder provides real-time speed and position feedback to support mobility control of the robot. The interrupt pins are **2** and **3** in the Arduino Mega for the both encoders Channel A.

2.2.5 MOTORS

Electric motors play a vital role in robotic systems by transforming electrical Energy into mechanical movement, which drives the operation of different moving parts within the robot. This project used two types of DC motors to perform specific tasks. This involves using two types of motors: the Maxon RE40 brushed DC motor and the Faulhaber 3557 012 CS coreless DC micro motor. Each motor was chosen to suit the specific functional requirements of different robot parts, one for high-torque applications and the other for precise, compact control.

2.2.5.1 MAXON RE40 BRUSHED DC MOTOR

The tracked robot will utilize the Maxon RE40 DC motor for the movements. It is a brushed DC motor, and Maxons are used in industrial, automation, robotics, mobility operations, etc. The Maxon RE series offers high efficiency, long life, and good torque output. In this project, they will be assisting the robot in traversing uneven ground and overcoming obstacles.

Figure 2: Maxon RE40 brushed DC motor [7]

This motor features **a 40 mm diameter** housing and delivers continuous torque of around 85 Nm, with peak torque reaching much higher values depending on the current input. This motor is an ironless rotor designed to help reduce cogging and give better control and smoothness at low speeds. This advantage is helpful for applications requiring fine-tuned response tasks like climbing steps, etc.

The motor runs efficiently at voltages ranging from **12 to 48 V**, and in this setup, it is operated at around 24 V for balanced speed and torque characteristics. The no-load speed is approximately **10,000 rpm**, though it is typically geared down through a reduction gearbox in the robot to provide more usable torque and lower wheel speed. The Maxon RE40 DC motor is excellent for heat dissipation and is also capable of longer operations without overheating.

Motor data				
	Values at nominal voltage			
1	Nominal voltage	V	12	24	48
2	No load speed	rpm	6920	7580	7590
3	No load current	mA	241	137	68.6
4	Nominal speed	rpm	6380	6940	7000
5	Nominal torque	mNm	94.9	177	187
6	Nominal current (max. continuous current)	A	6	6	3.17
7	Stall torque	mNm	1720	2420	2560
8	Stall current	A	105	80.2	42.4
9	Max. efficiency	%	88	91	92
	Characteristics				
10	Terminal resistance	Ω	0.115	0.299	1.13
11	Terminal inductance	mH	0.024	0.082	0.33
12	Torque constant	mNm/A	16.4	30.2	60.3
13	Speed constant	rpm/V	581	317	158
14	Speed/torque gradient	rpm/mNm	4.05	3.14	2.97
15	Mechanical time constant	ms	5.89	4.67	4.28
16	Rotor inertia	gcm²	139	142	137

Figure 2: Data of Maxon RE40 brusher DC motor [8]

Electrically, this motor pairs well with encoder feedback (like the ME22 series encoders used here), making it suitable for closed-loop PID control of speed and direction. Its compact size and high output power make it ideal for mobile robotic platforms where space and weight are constrained, but performance cannot be compromised.

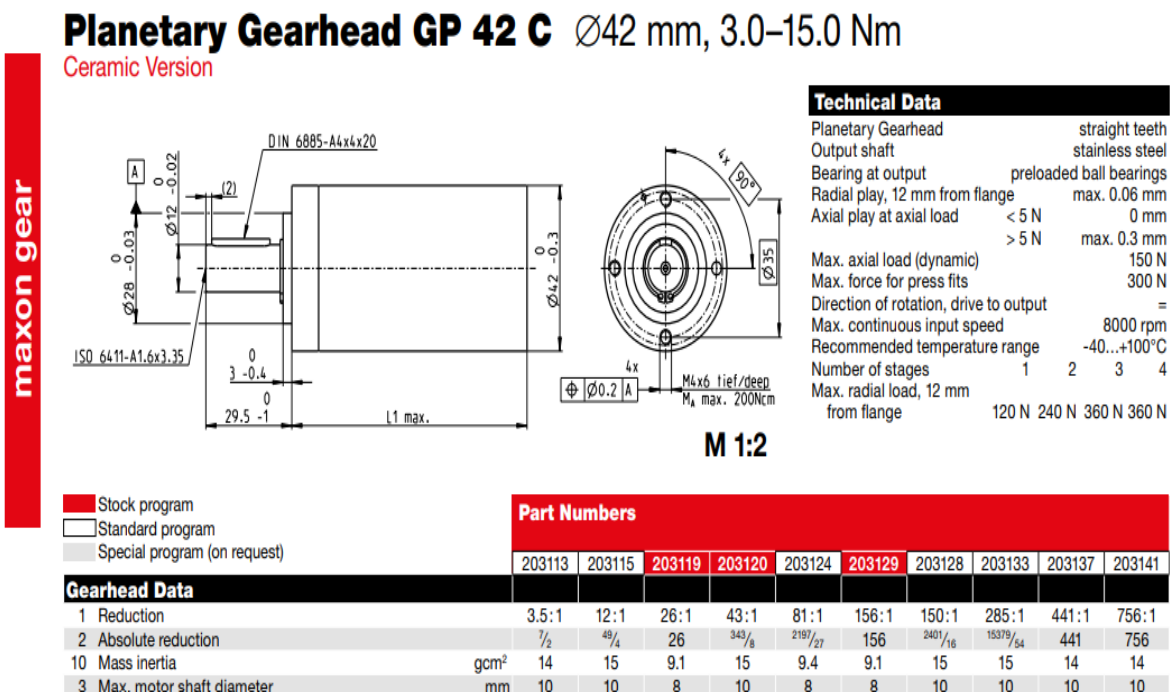


Figure 2: Data of the *GP 42 C planetary gearbox [8]*

In addition to the motor, the **GP 42 C planetary gearbox [Part Number: 203129]** is used in a ceramic version, which offers a reduction ratio of **156:1**.

2.2.5.1 FAULHABER SERIES 3557 CS DC MICROMOTOR

Compared to the larger and heavier RE40, the **Faulhaber 3557 012 CS** is a much more compact and light-weight **coreless DC micro motor**. It is an excellent choice for precision applications where limited space and fine control are two primary concerns. The 3557 is used in the flipper arm mechanisms of the robot, where the ability to move accurately with low inertia and a fast response time is prioritized over raw torque.

Figure 2: Faulhaber 3557 012CS brushed DC micro motor [9]

The 3557 has a diameter of 35 mm and a length of just under 60 mm, making it the perfect small package to integrate into compact modules with excellent performance. It uses Faulhaber's coreless rotor technology and specifically the design is an ironless rotor, which replaces the traditional iron core in the rotor with a lightweight, iron-free rotor.

Serie 3557 ... CS

		3557 K		009 CS	012 CS
1	Nennspannung	U_N		9	12
2	Anschlusswiderstand	R		0,7	1,34
3	Abgabeleistung	$P_{2\,max.}$		28,1	26,1
4	Wirkungsgrad, max.	$\eta_{max.}$		78	79
5	Leerlaufdrehzahl	n_0		5 700	5 400
6	Leerlaufstrom (bei Wellen ø 4 mm)	I_0		0,19	0,125
7	Anhaltemoment	M_H		188	185
8	Reibungsdrehmoment	M_R		2,8	2,6
9	Drehzahlkonstante	k_n		643	456
10	Generator-Spannungskonstante	k_E		1,56	2,19
11	Drehmomentkonstante	k_M		14,9	20,9
12	Stromkonstante	k_I		0,067	0,048
13	Steigung der n-M-Kennlinie	$\Delta n/\Delta M$		30,3	29,2
14	Anschlussinduktivität	L		100	220
15	Mechanische Anlaufzeitkonstante	τ_m		16	16
16	Rotorträgheitsmoment	J		50	52
17	Winkelbeschleunigung	$\alpha_{max.}$		37	35

Figure 2: Data of Maxon RE40 brusher DC motor [10]

This keeps the rotor extremely lightweight, allowing the motor to change speeds and the rotor's direction quickly. This is an excellent benefit for robotic arms, requiring them to move fast and accurately.

In this existing application, the user should use a **three-channel encoder (like the HEDS-5540)** to maintain positional feedback. This Encoder will allow for closed-loop control and not just allow this motor to move with an angular distance or position. The user could program for its speed too, which is valuable for robotic arms that must perform manipulation tasks or adjust posture with sensor feedback for adjustment to their intended task.

Series 32/3 S

	32/3 S
Housing material	metal
Geartrain material	steel
Recommended max. input speed for:	
– continuous operation	4 000 min^{-1}
Backlash, at no-load	≤ 1 °
Bearings on output shaft	ball bearings, preloaded
Shaft load, max.:	
– radial (10 mm from mounting face)	≤ 200 N
– axial	≤ 200 N
Shaft press fit force, max.	≤ 250 N
Shaft play	
– radial (10 mm from mounting face)	≤ 0,03 mm
– axial	≤ 0,15 mm
Operating temperature range	- 20 ... + 125 °C

Specifications						
Number of gear stages		2	3	4	5	5
Continuous torque	Nm	7	7	7	7	7
Intermittent torque	Nm	10	10	10	10	10
Mass without motor, ca.	g	190	230	260	290	300
Efficiency, max.	%	80	70	60	55	55
Direction of rotation, drive to output		=	=	=	=	=
Reduction ratio [1]		14:1	43:1	134:1	415:1	1 526:1
(rounded)		23:1	66:1	159:1	592:1	

Figure 2: Data of the 32/3 series planetary gearbox [11]

The motor is used with a **Faulhaber 32/3 series planetary gearbo**x, which has a reduction ratio of **246:1**. An incremental encoder from the HEDS 5540 C series measures the movement. [11]

2.2.6 BRAKES

Brakes serve an essential function of safety, control, and stability. For this project, the brake is an active safety component to restrain the flipper arms in the desired homing position. This brake is an electronic brake which provides a more accurate operation. The brakes help lock the position of the flipper arms and the drive tracks, adding a layer of safety for where they can go.

The electromagnetic brakes are compact and are electronically controlled devices that engage or release the brakes according to when a voltage can be applied or cut off. When voltage is applied to the brake, the brake is released and vice versa if the voltage is cut off.

Parameter	Value
Power Supply	12 V
Resistance	24 Ω
Current Consumption	0.5 A
Power Consumption	6 W
Coupling Time	13 ms
Separation Time	27 ms
Static Torque	400 Nm
Moment of Inertia	10 g·cm²

Figure 2: Data of the Electrical Brake

This system has some constraints:

Response Time: Brakes need to engage or disengage promptly or synchronize with the control logic.

Load Capacity: Brakes should be sized to withstand the generated torque of the motors and the external loads.

Electrical Control: A device to control the brake must transition straight to the point of braking without delay or excessive heat dissipation.

2.2.7 MOSFET

A ZVN4206A N-channel MOSFET controls the electromagnetic brakes. This part is a low-side switch that turns on and off the brake coil upon receiving an order from its Arduino Mega 2560 microcontroller.

Figure 2: MOSFET-ZVN4206A [12]

Working Principle:

Brake Dropped: When the microcontroller drives a MOSFET gate HIGH, conduction occurs on a single side of your brake coil circuit, and you hear a click. The electromagnetic force in the brake operates against that of a spring to release the brake on rotation of an associated motor.

Brake On: Driving the MOSFET gate LOW switches OFF this device, breaking current to the brake coil. The inner spring of the brake comes into contact with the friction pad and is pressed against the rotor to stop the motor shaft instantly.

PARAMETER	SYMBOL	VALUE	UNIT
Drain-Source Voltage	V_{DS}	60	V
Continuous Drain Current at T_{amb}=25°C	I_D	600	mA
Pulsed Drain Current	I_{DM}	8	A
Gate-Source Voltage	V_{GS}	± 20	V
Power Dissipation at T_{amb}=25°C	P_{tot}	0.7	W
Operating and Storage Temperature Range	T_j:T_{stg}	-55 to +150	°C

Figure 2: Absolute Maximum Ratings of ZVN4206A [13]

2.2.8 ULTRASONIC SENSOR

The SRF02 ultrasonic sensor is a range-finding module for measuring distance, detecting objects, and avoiding obstacles. The principle of this sensor is that it emits a high-frequency sound wave and receives the echo, which bounces back from an object, and measures the time taken to calculate the distance of an object from the sensor.

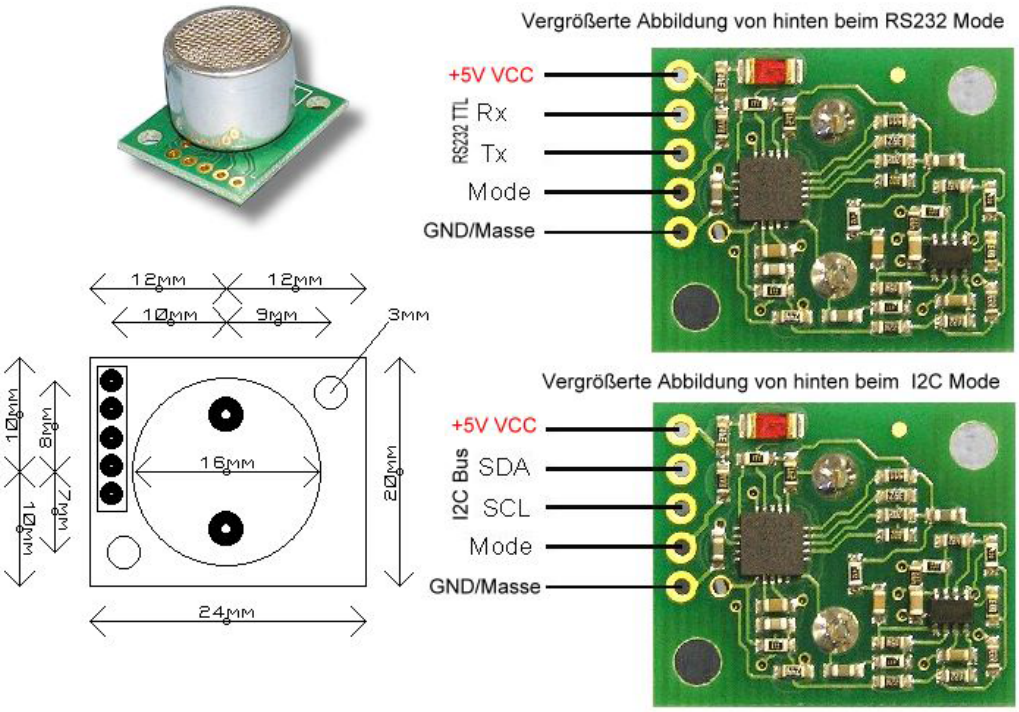


Figure 2: SRF02- Ultrasonic Sensor Pin out [14]

Working Principle

The SRF02 functions with a single transducer handling both the sending and receiving of the ultrasonic pulses. Roughly 40 kHz of ultrasound is emitted by the sensor. Sound waves striking an object reflect to the transducer. The sensor measures the time of flight of the echo and uses the speed of sound in air to make distance calculations.

Advantages:

- It maintains a non-contact form of distance measuring, making it suited for sensitive and moving objects.
- Covers an extensive range, detecting distances from 16cm to 6m.
- Output can be in centimetres, inches, or microseconds, enabling easier system integration.

Parameter	Value
Operating Voltage	5 V
Average Current Consumption	4 mA (typical)
Maximum Current (during ping)	15 mA
Operating Frequency	40 kHz
Range	16 cm – 6 m
Beam Width	55°
Communication Modes	I²C or Serial (UART)
I²C Address Range	0x70 to 0x7F
Measurement Units	cm, inches, or µs

Figure 2: Technical Specifications of SRF02- Ultrasonic Sensor

The SRF02 has two communication modes:

I²C Mode

Allows for the integration of multiple sensors on a single two-wire bus. Each sensor can be assigned a unique programmable I²C address. Best suited for robotics systems that need multiple range detections at a time.

Serial (UART) Mode

Utilizes a standard TTL-level UART as a point-to-point link interface. Easy to use for systems with a single sensor. For your tracked robot design, SRF02 sensors are I²C connected, which allows multiple sensors to share a single bus for front and rear obstacle detection.

Standard Use Cases within the Scope of the Project

As part of the mobile robotics design, there are five integrated SRF02 sensors. Three sensors are located on the front of the robot, while two are on the rear. Front sensors detect obstacles, and the rear sensors are for collision detection.

Each SRF02's data and information from the encoders and brakes form a closed-loop control system for achieving accuracy and navigation without collisions. The front sensors command the flipper arms to alter their position and angle during obstacle traversal, while the rear sensors help avoid obstacles during reverse maneuvers.

2.2.9 MULTI-CONDUCTOR TERMINAL BLOCKS

The terminal blocks are the most important component in this project because they give a compact and safer method to connect many wires with the same potential. The model XBUT series is used in this project for its connectivity and easy installation. The XBUT series is specifically designed for high-density wiring situations where multiple connections must be combined without additional bridging components.

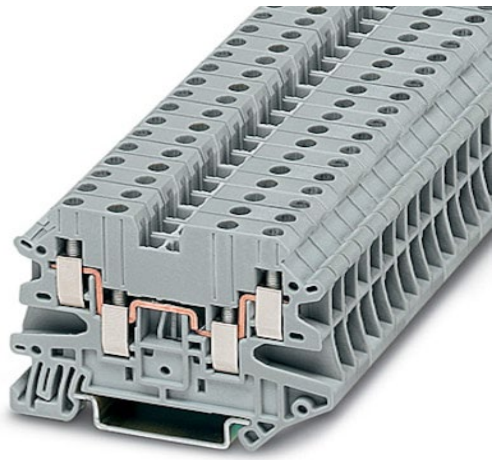


Figure 2: Terminal Blocks- XBUT

These terminal blocks are built using tin-plated copper alloy for the conductive parts that verify the low contact resistance and protection against corrosion. Also, the insulating body is designed according to IEC and UL standards. The terminal blocks are mounted on 35 mm DIN rails, allowing for easier installation and maintaining a stable hold with the control panel. Depending on the model, the terminal blocks support cross-sections from 0.14 mm^2 to 4 mm^2, and can handle currents up to 39 A.

The highlight of this series of terminal blocks is its flexible plug-in system, which allows the terminals to be interconnected without any external wiring. Using terminal blocks makes the electrical setup more reliable and improves wiring.

In this project, the 12V battery is the main power supply, which 12V power supply is given to the terminal blocks; only motor drivers and brakes operate at 12V. The remaining hardware operates at 5V. To overcome this, a DC-DC converter is used to convert 12V into 5V, so in the entire terminal blocks, only a few are 12V, and the remaining few are 5V for the Hall Effect sensor, Ultrasonic Sensor, and encoders.

2.2.10 DC-DC CONVERTER

A DC-DC converter is one of the main components for every system that needs multiple voltage levels for its subsystems. This project uses an LM2596S-based DC-DC buck converter to step down a 12V power supply to 5V for a few hardware components like ultrasonic sensors, Hall sensors, and encoders. This converter is a non-isolated step-down voltage regulator.

Figure 2: LM2596S DC-DC Converter [15]

The primary function of this converter is to convert the higher DC voltage to a lower, stable DC output without any heat dissipation because it is more important for this kind of battery-powered robots that should rely on energy consumption efficiently for longer operational time.

Parameter	Specification
Input Voltage Range	4.5 – 35 V DC
Adjustable Output Voltage	1.25 – 30 V DC
Typical Application	12 V to 6 V step-down for control electronics
Rated Output Current	2 A continuous, 13 W maximum output power
Efficiency	Up to 92% (depending on load and voltage difference)
Switching Frequency	~150 kHz
Voltage & Load Regulation	±0.5%
Output Ripple	100 mV max (at 12 V input to 5 V / 2 A output)

Figure 2: Specifications of LM2596S DC-DC Converter

Working Principle:

The LM2596 runs as a switching buck converter; it specifies that it reduces voltage through high-speed switching and does not dissipate excessively like the linear regulator. The basic mechanism of this converter is given as follows:

Switching and Energy storing:

This module rapidly switches current through an inductor and a diode at nearly 150 KHz. The energy is stored in the inductor when the switch turns ON.

Energy releasing:

If the switch is turned OFF, this stored energy discharges to the load via the inductor. The capacitors will filter the voltage and provide a stable DC output.

Potentiometer:

The built-in potentiometer enables the output voltage to be set anywhere from 1.25V to 30V, which provides the module with versatility to deliver power to the hardware components.

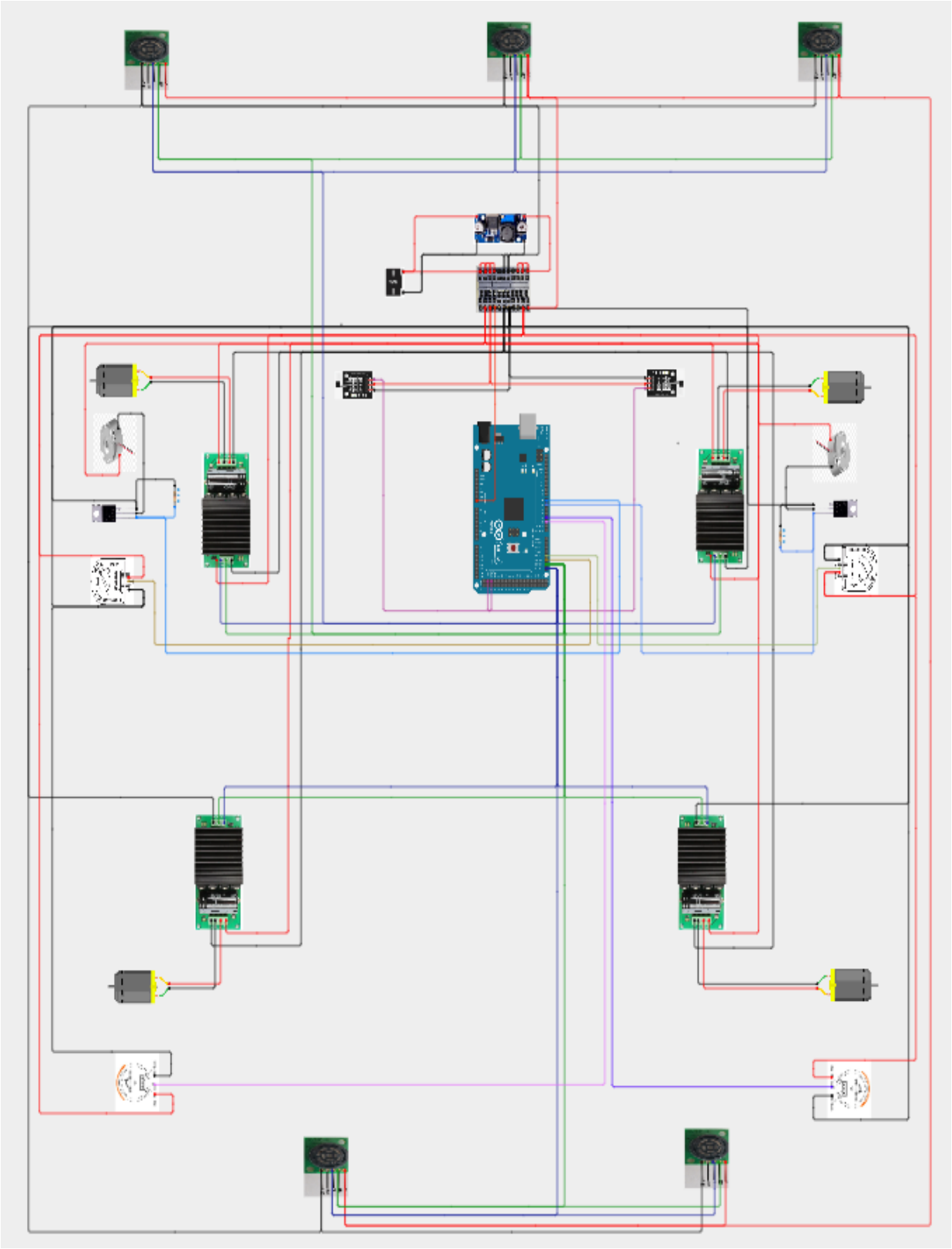
Integration in the robot system:

In this mobile robot, the **12 V battery pack** powers the **motors and brakes** directly because they require higher current at the native battery voltage. However, **control and sensing electronics** like the **Arduino Mega 2560, Hall sensors, Encoder, and ultrasonic sensors** require a **lower and stable voltage (around 6 V or 5 V)**.

The LM2596 module is placed between the **battery and the low-voltage bus**. It steps down 12 V to whatever voltage is needed. Overall, the role of the module is the heart of the low-voltage power system. Some important things to notice:

1. **Continuous Power supply:**
    - It ensures that the microcontroller and the sensors do not reset during motor start or sudden load change.
2. **Noise Reduction:**
    - The low output ripple allows for low noise output, allowing stable readings for the ultrasonic sensors and avoiding interference in communication lines.
3. **Energy Efficiency:**
    - Typically, this converter minimizes energy loss thus, allowing the robot to operate longer on a single charge.

2.3 WIRING CONNECTION



2.4 HALL SENSOR MOUNT DESIGN

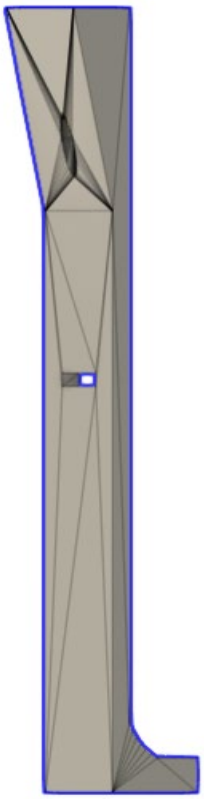


Figure 2: Side view of hall sensor beam

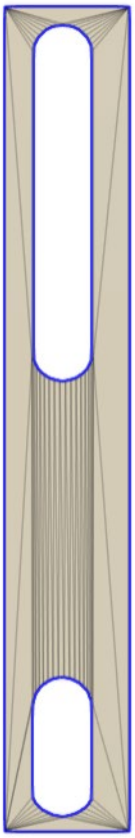


Figure 2: Top view of hall sensor support

The Hall Effect sensor is placed close to the magnet attached to the rotating shaft of the motor that operates the flipper arm. The sensor must be placed in a stable position, so the hall sensor beam, and support plays an important role, the bottom of the hall sensor beam is attached to the base of the tracked robot by screwing it that makes the strong base, where the hall sensor support as shown in the figure is attached to the top of the beam, and the other end is connected to the tracked robots body. The sensor is placed in the exact position, which is close to the magnet, to detect without any delay. This design holds the sensor steady even in different real-world conditions.

2.5 COMMUNICATION INTERFACES

In this robotic system, the communication interface **links the controller and its peripherals**, allowing smooth data exchange between sensors, motor drivers, and the microcontroller. Here, I2C communication is used to connect to the digital components.

For this project, we pick communication interfaces carefully based on speed, reliability, and compatibility with the hardware. The Arduino Mega 2560, SRF02 ultrasonic sensors and motor drivers monitoring circuits, are all run together using available standard digital communication protocols.

Figure 2: I2C message

I2C Communication Protocol Operation

I²C Communication Protocol Data Transmission Flow:

1. Addressing Phase

Here, the controller starts communicating by sending the 7-bit or 10-bit address of the target device on its Serial Data Line (SDA). The latter mechanism guarantees that future commands only act on the intended peripheral.

2. Read/Write Indication

After the address, a single bit is sent that specifies which direction the operation is going in:

- **Logic 0 (Write)**: Controller writes data to the peripheral
- **Logic 1 (Read):** Controller takes data from the peripheral

3. Data Transfer with Acknowledgment

Data is exchanged in 8-bit packets along with an acknowledgement (ACK) bit. However, the receiving device does not acknowledge this data till the ninth clock pulse, when it pulls SDA low. The controller detects a transmission error through the absence of an acknowledgment (NACK). The Handshake Protocol ensures the reliable transfer of data by doing this before continuing to more bytes.

4. Clock Synchronization

This interface uses a Serial Clock Line (SCL) for critical Timing Synchronization:

- **Rising edge:** when devices sample data
- **Falling edge:** to indicate that data is allowed to change

The purpose of this clock is to keep the controller and all peripherals in exact synchronization with each other so peripherals can correctly interpret the address boundaries, data segments, control signals, etc.

The above combination of addressing, direction control, acknowledgment verification, and clock synchronization provides a robust serial communication architecture for multiple-device systems.

Overview of I²C Communication

I²C is a synchronous serial communication protocol that allows multiple devices to communicate over a two-wire bus.

SDA (Serial Data Line): Transfers command and sensor data between I2C devices.

SCL (Serial Clock Line): Maintains communication timing integrity across the bus.

Here, the Arduino Mega 2560 acts as the master that initiates all the communication and also controls the clock. The hardware components, like motor drivers and ultrasonic sensors, act as slaves; each slave has its unique address.

When the master wants to communicate, it sends the slave address on the bus, and the device with that address responds.

Hardware Components	Address
Left Flipper arm motor Driver-MD03	0x58
Right Flipper arm motor Driver-MD03	0x59
Left Rear Motor-MD04	0x5A
Right Rear Motor-MD04	0x5B
Front SRF02 sensors	0x76, 0x7E, 0x74
Rear SRF02 sensors	0x73, 0x75

Figure 2: I2C address

To start the measurement of the ultrasonic sensors, a measurement command must be written to the command register. Different modes are distinguished, as the sensor can measure the required time and distance (cm or inches). The measurement result is written to two registers, which can then be read sequentially as shown in the figure.

The control of the motor drivers works similarly. By setting the bits in the individual registers, motor control is easily achieved. The drivers have a command register distinguishing forward, reverse, and stop. The PWM control value for the speed, acceleration, and direction of the designated motors can be written with their corresponding registers as shown in the figure.

3. CONTROL SYSTEM

The control system is the tracked robot's primary intelligence, responsible for controlling the movements, reading the sensor values, and the decision-making processes. The control system also verifies that the tracked robot runs safely and smoothly, and also processes data and sends commands to the motors to do their task as the user planned.

In this project, Arduino Mega 2560 is the heart of the controlled system, which executes the algorithm programmed by the user. A closed-loop structure is followed here to adjust the tasks continuously in real time. The main elements involved in this controlled system are:

Arduino Mega 2560 microcontroller:

- Acts as the brain of the robot, running the main control algorithms.
- Handles communication with SRF02 ultrasonic sensors via I^2C to detect obstacles.
- Sends control signals to the motor drivers (MD03/MD04) to regulate the speed via I^2C.
- Controls the MOSFET-based brakes for stopping in the desired homing position of the flipper arms.

Sensor Feedback:

- The SRF02 sensors continuously measure the distance to objects, which is important for detecting objects.
- Motor encoders and Hall sensors give information about the speed and position feedback that perform accurate movement control.

Motors and Brakes:

- With the help of the processed sensor data, the Arduino Mega 2560 instantly adjusts the speed and applies or engages the brake when needed.
- The work done by the motors and brakes helps the robot move smoothly and control the speed and the desired homing position.

Combining all the above elements forms the backbone of the robot's control system and also boosts the system to react fast to the real-time obstacle detection. By combining all the elements mentioned above forms the backbone of the robots control system and also boosts the system to react fast to the real-time obstacle detection.

3.1 PID CONTROL FOR SPEED REGULATION

Speed regulation is an important requirement in many mechanical and electrical systems, mainly to control the motor's speed for stable operation under different environmental conditions. The PID (Proportional- Integral- Derivative) controller is one of the most optimal motor speed control methods.

A PID controller works within a closed-loop feedback system. Generally, the actual motor's speed is measured using a tachometer to detect the speed manually, the encoder pulses, or other speed-sensing devices. Here, the motor's speed is controlled using encoder pulses and a single channel to count the rising edge and compare it with the desired reference speed.

The difference between the desired speed and actual speed is called the error [16]. The calculated error is the input for the PID algorithm that calculates the control signal to alter the motor's input to keep the speed at the desired target.

The controller makes its decisions using three main building blocks:

Proportional term (P) – produces an output directly proportional to the current error. A larger proportional gain can speed up the response, and if set too high, it can cause overshooting.

Integral term (I) – Look at the error's history and add up past values. This helps the controller remove any steady, lingering difference between the desired and actual output, mainly when constant disturbances occur.

Derivative term (D) – Focuses on how quickly the error changes. This helps the controller remove any steady, lingering difference between the desired and actual output, especially when constant disturbances occur. [16]

The overall control function can be expressed as:

$$u(t) = K_p\, e(t) + K_i \int e(t)\, dt + K_d\, de(t)/dt$$

- e (t) = speed error
- Kp = proportional gain
- Ki = integral gain
- Kd = derivative gain

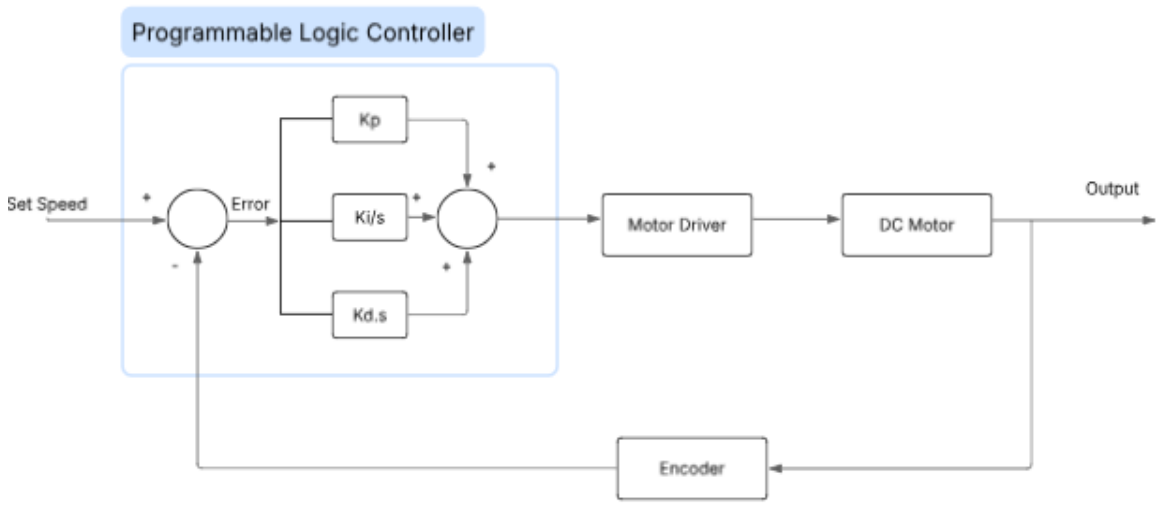


Figure 3: PID Controller diagram

The main purpose of the PID controller in this project is to maintain a constant desired linear speed **(0.04 m/s)** for both left and right drive motors in real-world disturbances. The maximum speed of the tracked robot can move up to **0.07 m/s, i.e., 4.2 m/min.**

Speed Calculation:

The Maxon RE40 motors have a **high-resolution encoder** that provides **23400 counts per wheel rotation (after the correction factor).** The Encoder signals from both motors are read through hardware **interrupts on pins 2 and 3** to ensure accurate and real-time pulse counting. The wheel circumference (0.105m diameter) converts Encoder counts into linear speed in meters per second.

Wheel Circumference C= $\pi \times D = \pi \times 0.105 \approx 0.3299$m

CPR = 150×156 =23400

Gear ratio= 45/32 =1.2

Time interval Δt = 100ms = 0.1s

The formula is:

$$Speed = \frac{Encoder\ per\ pulse}{CPR \times \Delta t \times Gear\ ratio} \times wheel\ Circumference$$

Once the speed is calculated, the measured speed is compared to the desired speed set by the user, i.e., 0.04 m/s, and a speed error value is given. The error is fed into a PID (Proportional–Integral–Derivative) control algorithm, which calculates an output signal proportional to the magnitude of the error, its accumulated sum over time, and the rate of change of the error.

The PID output determines the motor drive level required to correct the deviation in speed for maintaining the desired speed. To allow smooth acceleration and avoid sudden increases or decreases in motor power, the calculated output will then be processed through a smoothing filter and act as the base PWM command for the motors.

Once the base PWM value is found, the system makes minor changes to account for any speed differences between the left and right wheels. Implementing the low-pass filter verifies that the correction between the left and right wheels rotates at the same rate, which helps to maintain straight-line motion. The final values of PWM are sent to each wheel through the I2C interface to the respective motor drivers.

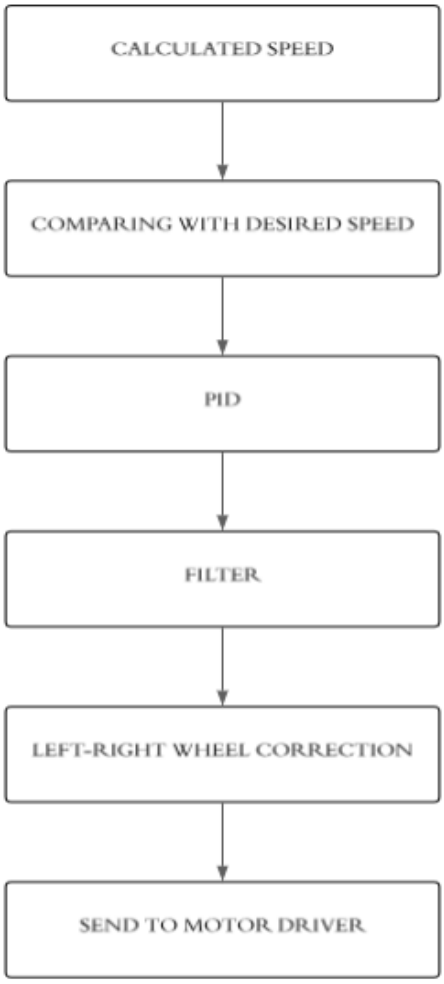


Figure 3: Speed-to-Motor Command Conversion Flowchart

3.2 ENCODER INTEGRATION FOR ANGLE CALCULATION

Rotary encoders and DC motors are being used in this project for position and velocity feedback. Each Encoder makes digital pulses that correspond to the movement (rotation) of the motor shaft, which is used to measure the system's linear displacement and angular position.

The Maxon RE40 motors for movements have incremental encoders connected to the Arduino Mega interrupt pins (2 and 3). The rising edge of each encoder signal is counted using interrupt service routines. In the code, countEncoderA and countEncoderB are for the Encoder counts, and those counts are stored in variables countA and countB. The instantaneous speed of each wheel is calculated by counting the change in count over a sampling period and relates to the incremental value: The angle calculation is given as follows:

Encoder CPR = 100

Reduction Ratio =246

$$Counts\ per\ Degree = \frac{Total\ CPR}{360} = \frac{24600}{360} = 68.33$$

$$Angle\ from\ Encoder\ count = \frac{Encoder\ count}{Counts\ per\ degree}$$

Once the angle from the encoder is calculated, then the user's desired input (target angle) to move the flipper arm is calculated using the counts per degree.

$$Target\ Angle = Angle \times Counts\ per\ degree\ \times 2$$

Here, 2 represents the gear ratio (2:1) between the shaft and the flipper arm.

Thus, any target angle can be accomplished by comparing the encoder count with the corresponding encoder ticks for the target. The function moveToAngle() uses the logic by moving the flipper forward or backward to a specific angle until the target angular position is achieved.

A Hall Effect sensor is mounted on the flipper arms at the home position to allow for repeatable and accurate angle measurement. When the system is turned on, the homeMotor() function will run the flipper until the Hall effect sensor is triggered, resetting the encoderCount to zero.

Integrating the encoder enables the execution of the function in this project:

Flipper Angle Control: Encoder counts and homing reference provide repeatable angular positioning of the flipper arms. By employing the encoders in this way, we ensure both the predictable linear motion of the robot concerning encoder counts and the consistent positioning of the auxiliary mechanisms.

3.3 FREQUENCY CALCULATION OF AN ENCODER SIGNALS

The encoder is a critical part of the system to obtain feedback on motion from the motor shaft. The encoder generates a number of digital pulses related to the rotation of the motor shaft. The frequency of this pulses are directly proportional to the rotational speed of the motor.

Since, we have the known values of encoder's counts per revolution, and the reduction ratio of the planetary gearbox and the maximum RPM of the wheel, we can calculate the frequency of an encoder signal with known inputs.

Frequency of an encoder signals from ME22-150-4.000-2LS:

Encoder CPR = 150

Reduction ratio = 156:1

Total CPR = 23400

Maximum RPM = 24.6 RPM

The formula is:

$$Frequency = \frac{Total\ CPR\ \times Maximum\ RPM}{60}$$

$$Frequency = 9{,}594\ HZ$$

Frequency of an encoder signals from HEDS-5540:

Encoder CPR = 100

Reduction ratio = 246:1

Total CPR = 24600

Maximum RPM = 20 RPM

The formula is:

$$Frequency = \frac{Total\ CPR\ \times Maximum\ RPM}{60}$$

$$Frequency = 8{,}200\ HZ$$

3.4 MOTOR MOVEMENT LOGIC

The system's motor movement logic has two main basic functions successfully implemented:

Locomotion Control - to control the tracked robot's forward, backward, left, right, and stop movements.

Flipper Arm Control - position the flipper arms based on home and angle position.

Locomotion Motor Logic:

The robot is driven by two DC motors (Motor A and Motor B), both controlled by I2C communication with motor driver 0x5A and motor driver 0x5B, respectively. The motor controller leverages two Registers: a Direction Register (DIR_REG) for direction control and a Speed Register (SPEED_REG) for speed control.

When controlling the motors, the following motor states exist:

- Forward - Both motors are rotating forward.
- Reverse - Both motors are rotating backward.
- Left Turn - Left motor rotates backward; right motor rotates forward.
- Right Turn - Left motor rotates forward; right motor rotates backward.
- Stop - Both motors are told to stop.

The function handleCommand () gets called to handle the serial input commands (f,b,l,r,s) on how to set the correct motor directions. A PID-based closed-loop control adjusts the PWM speed, controlling each motor to match the target velocity (0.04 m/s). Accounting for the speed differential will be compensated through the feedback from each motor's encoders.

Flipper arm motor logic:

The flipper arms are actuated using an MD03 motor driver at I2C address 0x59 and 0X58. The motor driver is paired with an encoder, allowing the angle to be accurately determined.

Homing sequence: At startup, the homeMotor() routine runs and drives the motor until the Hall effect sensor is "triggered", allowing the encoder count to be set to zero (this becomes the reference angle).

Angle movement: In order to move to a specified angle, the moveToAngle() function calculates how many encoder counts are needed to get to that angle. By driving either forwards or backwards, the motors will continue to run until the current encoder counter value is equal to the target angle. Once the angle is reached, we stop the motors and apply the brake logic.

Brake logic: o apply the brake using the applyBrake() and releaseBrake() functions, we use an N-channel MOSFET. The brake logic allows us to maintain position (where there ought to be no movement) but not allow the motors to rotate in case there is a disturbance.

Speed and Correction Control

The loop continuously calculates wheel velocity (speed) from encoder counts. Then, a PID controller generates PWM values to maintain the target velocity. The difference between the two motor speeds (speedA - speedB) is then used to correct side drifting between the left and right motors by applying a small correction factor that ensures the robot can walk straight.

Summary of Motor Logic:

- Serial command input will determine the motion modes.
- A PID-based closed-loop provides stable speed tracking.
- Homing will provide a reference for the flipper position.
- Encoder robot feedback, provided by both wheels and flippers, would ensure you are moving accurately.
- Braking would allow for a safe way to hold that position.

This layered control logic combines perfect robot motion, good linear locomotion, and angle-based flipping, which is only used in the flippers.

4. IMPLEMENTATION AND SETUP

4.1 SYSTEM SETUP AND CONFIGURATION

The implementation phase began with the physical assembly of the hardware components and configuration of the Arduino Mega, which would function as the central controller. Many devices make up the system, each serving its roles and responsibilities:

Locomotion motors (Motor A, Motor B):

The two DC motors are connected via I2C motor drivers with the I2C addresses of 0x5A and 0x5B. The motors function to drive the robot forward, backward, and turn.

Flipper arm motor:

This motor is dedicated to the flipper arm and controlled via an I2C motor driver: MD03; I2C address 0x59 and 0X58. The motor has an incremental encoder to measure angular displacement in degrees.

Encoders:

The encoders are attached to DC motors, feedback is provided to the interrupt pins on the Arduino. The encoders' measure pulses from each motor shaft rotation to provide instantaneous position feedback for speed and angle calculations.

Hall Effect sensor:

The Hall Effect sensor connects to one of the Arduino pins on the Arduino to determine the reference position of the flipper arm when homing. Once the flipper motor triggers the Hall Effect sensor, the relative encoder position is reset to zero, ensuring that gravity, friction, momentum, or any other variable does not affect the motor's angular measurement.

Brake circuit:

An N-channel MOSFET applies or releases the brakes on the flipper motor when an action is undertaken. The brakes act in the opposite direction of the force generated by the flipper arm; when in the parking position, the brakes provide a safe, stable hold when no flipper motor is being operated.

Serial Communication:

Provides a means to transmit user inputs to the system and receive state updates from the system through serial communication at a baud rate of 115200.

Configuration has been considered with respect to pin assignments and interrupt handling. The Arduino Mega could handle multiple interrupts, which we needed to enable proper, consistent capture of encoder pulses while controlling the motors.

A complete software configuration was considered through many functional modules addressing all desired functionality: motor direction control, motor speed control, encoder feedback, home routines, flipper angle position, PID speed control, and serial commands.

Each module will be researched and developed as an independent function, and then the functions will be combined into the main Arduino program. A modular approach for development facilitates debugging and enhancements in the future.

4.2 ARDUINO setup () FUNCTION

The setup () function is important because it sets up both the hardware and software environment to correctly transition to a continuous operating mode for the robot. The setup () function is called only once, when the system is started up, and in this project, it performs the following operations:

Communication Setup:

Serial communication is set up at (115200) baud to send commands to the microcontroller and monitor feedback from the robot using the serial terminal.

I2C communication (Wire.begin ()) is started for communication with the MD03, MD04, and SRF02 ultrasonic sensors.

Input/output Setup:

The encoder pins (ENCODER_A_PIN, ENCODER_B_PIN, and encoder_ChA) are set to detect the rising edges from the encoder.

The brake pin (brakePin), which will control the brake circuit of the flipper motor, is set to output. The mechanical brakes are connected to MOSFET for controlling when to apply and release the brakes.

Interrupt Setup:

The external interrupts are attached to the encoder pins; each time an encoder pulse comes in, the encoder interrupt service routine (countEncoderA, countEncoderB, updateEncoder) will increase the count variables, allowing us to measure how the motor shaft is being turned, even at higher speeds.

Initial Motor State:

We issue an initial stop command to avoid the robot's unexpected motions when the power is turned on.

The brakes will be released for a moment while the homing process occurs.

Homing Process for Flipper Arm:

At boot time, the command homeMotor () is called. The flipper motor will spin until the Hall Effect sensor detects the reference position. At this time, the encoder count will be reset to 0, so we always have the same home position for calculating all angles.

Once homing is done, we may lock the flipper arm down, using the brake, to maintain the home position.

By the end of the setup () function, the system is fully calibrated and ready to perform tasks. Everything in the hardware is initialised, the communications are established, and the flipper arm is set at zero reference. So, we are assured that the robot is initialized in a known and stable state.

Pin Configuration:

PIN NAME (IN CODE)	ARDUINO PIN NUMBER	FUNCTION
ENCODER_A_PIN	2	Motor A encoder signal input
ENCODER_B_PIN	3	Motor B encoder signal input
encoder_ChA	19	Left Flipper arm encoder (Channel A)
encoder_ChB	18	Right Flipper arm encoder (Channel A)
brakePin	7	Brake control (flipper arm)
brakePin1	6	Second brake control (flipper arm)
hallSensorPin	52	Left Hall sensor input for homing
hallSensorPin1	50	Right Hall sensor input for homing
ENCODER_A_PIN	2	Motor A encoder signal input
SDA	20	I²C clock line for motor drivers
SCL	21	I²C clock line for motor drivers

Figure: Tabular column for Pin configuration

In the above mentioned tabular column, the pin configuration is stated as declared in the code and pin configurations are exactly added as the connections that are done to the Arduino Mega 2560 microcontroller. The I²C communication in the tabular column is only for the reference where in the real code it is used with **writeI2C ().**

4.3 ARDUINO loop () FUNCTION

The loop is responsible for the continual handling of commands, continually updating encoder feedback, and applying control logic. The execution path can be summarized as follows:

Serial Command Processing

- The loop first looks to see if any serial input is available.
- Single-character commands (f, b, l, r, s) are read by handleCommand() to set both motor directions.
- Flipper commands (flipperAngle:X) are interpreted and extract the desired relative angle. Then, it computes the target encoder count and drives the flipper motor until the desired position is reached. Then, the motor is stopped and the brake is reapplied.

Encoder Feedback and Speed Calculation

- Every 100 milliseconds, encoder counts are read for every wheel to calculate wheel rotations and linear speeds.
- Distance travelled is converted to speed (circumference of the wheel divided by time elapsed).
- Convert the displacement to speed by using wheel circumference and elapsed time.

PID Speed Control

- The error is the difference between the target speed (0.04 m/s) and the average speed.
- A PID controller computes the correction value utilizing the proportional, integral, and derivative terms.
- A filtering step was included to average sudden jumps in PWM output.

- Calculate the speed difference between the left wheels and right wheels and correct with a factor to produce straight motion.

Motor Command Update

- The computed PWM values were written via I2C to both motor drivers.

If the stop command was given, the PWM values were set to zero.

Real-Time Data Logging:

For each cycle, the target speed, left wheel speed, and right wheel speed are sent to the serial monitor for the user to observe a record of performance and help with tuning of the PID gains.

Execution Flow

Because of the unique loop () structure used here, the robot will be able to respond to user demands while at the same time maintaining accurate control over the motors. Commands will be processed in real-time, and the closed-loop feedback will ensure the system goes at the intended velocity and the required angular position.

The loop () will also ensure that the performance is being monitored continuously, in case a discrepancy needs to be addressed. So, what does all this lead to? Smooth, predictable, and confident behaviour.

5 CONCLUSION AND FUTURE WORK

5.1 SUMMARY OF WORK COMPLETED

This work demonstrates the recommissioning and development of a tracked-driving robotic platform with control of a controlled flipper arm. This project focused on assembling and implementing various pieces of hardware - DC motors, motor drivers, encoders, a Hall Effect sensor, and a braking system - into the Arduino Mega, and integrating with its controlled mode.

In our implementation, we systematically followed a workflow. We had an initial visit on system configuration, which included configuring the hardware, establishing the communication links for the motor drivers (using I2C), and ensuring that the input/output links for encoders, Hall sensors, and braking systems were configured correctly.

Once we had established the configuration, we used a modular approach to our software architecture, with individual functions defining the header for homing, motor direction (forward and backward), encoder feedback, speed (calculated based on the change in encoder count), braking, and changing the position (angle) of the flipper arm. Breaking the functions into distinct modules made debugging and extensibility easier.

In terms of function, the setup() function, which was used on the Arduino, was used for initializing the system, attaching any interrupts to read the encoders, and performing a homing sequence to find the location of the flipper arm. The homing sequence is the most important sequence because it always gives the system a calibrated reference position.

The loop() function of the Arduino was in fact where the execution logic was implemented because both the loop() function and the setup() function was where the system was able to parse serial inputs in real time, calculate specified dues based on the encoder feedback, and perform closed loop control, with a PID controller, as a mean of controlling velocity.

Encoders were used for both speed control and angular positioning. In either case, the pulses counted on the encoders provided precise measurements, allowing calculations of linear wheel velocities and angular displacements of the flippers. The PID controller would allow smooth velocity tracking, and corrective logic would balance left and right wheel velocities, allowing for straight line travel.

This project successfully combined mechanical, electrical, and software components into a rudimentary working prototype for a robotic system. The implementation proved an ability to control a robotic system based on encoders, while also showcasing an important aspect of robotics: feedback systems provide a means to ensure reliable motion.

5.2 RECOMMENDATIONS FOR IMPROVEMENTS

Accurate Hall Sensor Orientation:

Make sure that the Hall sensor is orientated exactly in line with the flipper's hinge rotation. Even small deviations from this orientation can result in angle detection errors, which can affect repeatability. Utilize a guided mount or adjustable fixture to maintain proper repeatability.

Improved Motor Control PID Algorithm:

The currently implemented PID control will provide basic speed and position control, there is room for improvement with respect to response time, timing of motion, limiting overshoot, and controlling more dynamic loads, by exploring adaptive or feedforward control strategies.

Use of Stronger but Lightweight Materials to Reinforce Motion Arms:

Employing lighter but stronger materials (or alternate support structures) to create a decreased amount of flex and vibrations during motion will contribute to smoother action and longer life of all the moving elements.

Design for Replacing and Maintaining Modular Parts:

To make it easier to maintain (or to upgrade or replace the sensors in the future, if necessary), designing the flipper assembly as a modular system with separable layers of components would be much easier than having to disassemble the entire thing.

5.3 FUTURE ENHANCEMENTS

Wireless Data Transmission:

With the addition of wireless communication modules (Bluetooth, Wi-Fi, etc...), encoder signals and motor performance could be monitored remotely, reducing wiring and enabling diagnostics in real-time.

Integration of IoT:

Connecting the encoder system to an IoT platform would allow long-term performance tracking, maintenance forecasting, and optimization of operation based on data. Energy efforts would also be increased.

Energy Saving:

Energy-efficient motor control techniques and regenerative braking can help reduce the power consumption while adding to the overall sustainability of the system.

[1] Arduino Mega 2560 https://docs.arduino.cc/hardware/mega-2560/

[2] TLE4905L- utsource Hall sensor https://www.utsource.net/itm/p/5286360.html

[3] MD03 motor driver https://www.robot-electronics.co.uk/htm/md03tech.html

[4] Md04 motor driver https://www.robot-electronics.co.uk/files/md04tech.pdf

[5] HEDS/HEDM Encoder https://www.farnell.com/datasheets/1884440.pdf

[6] ME22-PU Encoder https://elektrosil-buehlermotor.de/media/pdf/e6/1c/29/PWB-encoders-Data-Sheet-ME22_PU_Rev-4_engl_01012020.pdf

[7]MaxonRE40motorimagehttps://www.maxongroup.com/maxon/view/product/motor/dcmotor/re/re40/148866

[8] MaxonRE40motor https://www.farnell.com/datasheets/1683370.pdf

[9]Faulhaber3557https://www.ebay.de/itm/255724470478?_skw=FAULHABER+3557K012C+DC+Micro+Motor&itmmeta=01K2X8XR9BMV4XXPHM9FS48MH2&hash=item3b8a5dc0ce:g:YbkAAOSwAo5jIF0V

[10]DataofFaulhaber3557https://www.glockenankermotor.com/media/pdf/be/a2/60/DE_3557_CS_DFF.pdf

[11] Data of planetary 32/3 https://www.faulhaber.com/fileadmin/Import/Media/EN_32-3S_FMM.pdf

[12]zvn4206 image https://www.reichelt.de/de/de/shop/produkt/mosfet_n-ch_60v_0_6a_1r_to-92-68631#open-modal-image-big-slider

[13]Dataofzvn4206mosfethttps://cdnreichelt.de/documents/datenblatt/A100/ZVN4206A.pdf

[14]Ultrasonic sensor https://www.mikrocontroller.net/attachment/564267/srf02doku_dr.pdf

[15]DC-DC Converter https://eckstein-shop.de/LM2596SDC-DCeinstellbarerStep-DownSpannungsreglerAdjustablePowerSupplyModul

[16]PID Control https://www.ni.com/en/shop/labview/pid-theory-explained.html